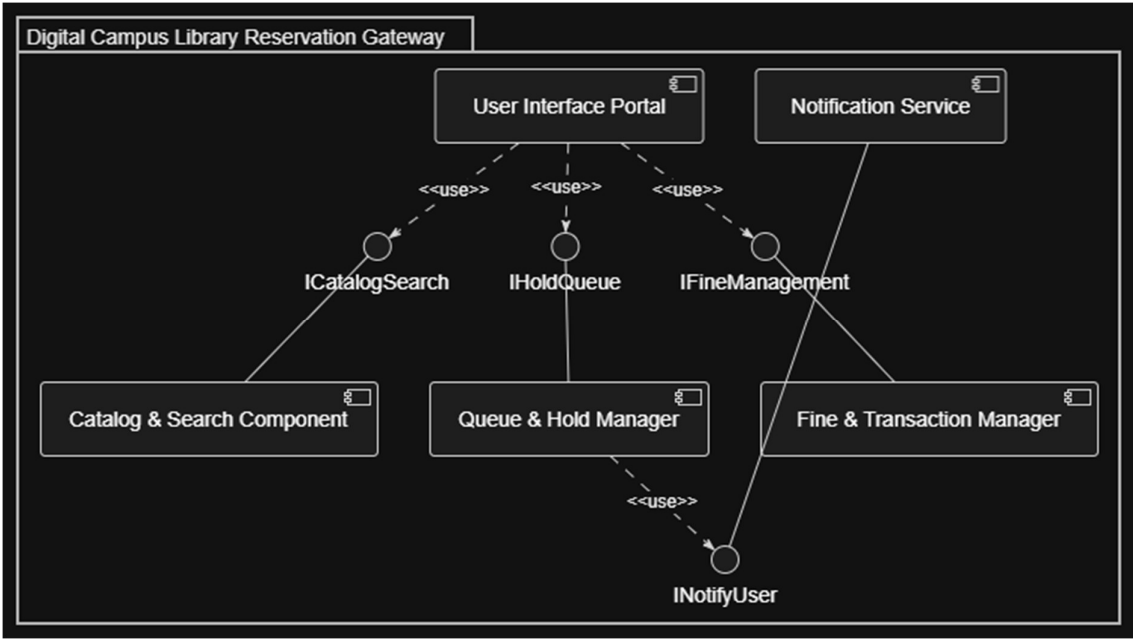


Software engineering Lab -03

Uml Diagram



Interface Requirements:

Interface Name	Provided By (Ball / Circle)	Required By (Socket / Semicircle)	Communication Protocol / Data Flow	Description & Component Interaction
ICatalogSearch	Catalog & Search Component	User Interface Portal	HTTPS / REST (JSON)	Data Flow: Search query (ISBN/Title) → Paginated book list. Interaction: User interface requests ISBN catalog lookup; Catalog component executes fast queries and returns metadata.

Interface Name	Provided By (Ball / Circle)	Required By (Socket / Semicircle)	Communication Protocol / Data Flow	Description & Component Interaction
IHoldQueue	Queue & Hold Manager	User Interface Portal	gRPC / Protocol Buffers	Data Flow: Book ID + Student ID → Reservation confirmation & FIFO Position. Interaction: UI sends hold request for loan item; Manager updates FIFO queue and returns position.
INotifyUser	Notification Service	Queue & Hold Manager	Asynchronous Message Queue (RabbitMQ / Kafka)	Data Flow: Event payload (User ID, Book Title, Pickup Window). Interaction: Queue manager detects book return and publishes an event; Notification service dispatches email/SMS.
IFineManagement	Fine & Transaction Manager	User Interface Portal	HTTPS / REST (JSON)	Data Flow: Student ID → Outstanding balance & Overdue transaction logs. Interaction: UI checks fine status during checkout/reservation; Fine Manager processes payment logs and updates ledger.

Justification

ARCHITECTURAL SELECTION & JUSTIFICATION

1. Architectural Selection

We chose Microservices Architecture for the Digital Campus Library Reservation Gateway.

2. Justification & System Analysis

Scenario-Related Reason 1 (Dynamic Traffic & Peak Load Handling):

Library systems experience severe traffic spikes during exam periods when thousands of students simultaneously search catalogs and place holds. Under a Microservices

architecture, the Catalog & Search Component and Queue & Hold Manager can be independently

scaled out using container orchestration without incurring unnecessary resource costs for less frequently accessed modules like Fine & Transaction Management.

Scenario-Related Reason 2 (Fault Isolation & System Availability):

In a monolithic design, a failure in third-party email/SMS gateways during hold notifications could crash the whole application. Microservices decouple these services via asynchronous event messaging. If the Notification Service or Fine Manager experiences downtime, the core

catalog search and book hold functionality remain fully operational for students.

3. Security Advantage

The Microservices pattern establishes strict network boundary isolation for sensitive operations.

The Fine & Transaction Manager component, which processes student financial records and fine

payments, operates within an isolated, encrypted subnet with restricted API gateway routes.

This prevents vulnerabilities in public-facing modules (like Catalog Search) from compromising

transaction logs or student financial data.

4. Performance Benefit

Microservices allow dedicated database patterns per service. The Catalog & Search Component

utilizes a read-optimized, in-memory cache (e.g., Redis) alongside an indexed search database,

enabling execution of paginated catalog queries across 50,000+ ISBN records in under 150 ms

(NFR-001) without being blocked by write locks from queue updates or fine calculations.